

Claude Code Mastery: 6-Week Syllabus and Roadmap

Level-Based Mastery Roadmap (Claude Code L1–L4)

L1 – Everyday PM & Productivity Workflows: The foundation is using Claude Code for day-to-day tasks and project management productivity. At this level, you learn core features: starting Claude in a project, using the chat agentic loop to edit code, run tests, and handle basic tasks. It covers writing a **CLAUDE.md** with project or team conventions (e.g. coding style, “always do X” rules) and using built-in tools safely ¹. Expect to draft product specs in plain English and have Claude generate or refine code from them, summarize documents, and automate simple chores. *Examples:* generate a user story outline or PRD from a prompt, use Claude to analyze a code snippet or log, or create a quick Python script for a personal budget calculation. Mastery of L1 means Claude Code is part of your daily workflow – you can ask it to build features, debug issues, navigate your codebase, or even summarize project info on command ². At this stage, **permissions** are used in default safe mode (Claude asks before file writes or shell commands) to prevent accidents.

L2 – Claude Code Power Features: Level 2 introduces advanced extensions for power users: **Skills**, **Subagents**, **MCP** (Multi-Channel Providers) connectors, **Hooks**, and **Plugins**. You learn to extend Claude’s knowledge with project-specific Skills (Markdown files of reusable knowledge or task workflows) and to invoke them via slash-commands. You configure custom Subagents – isolated AI helpers in `.claude/agents/` that Claude can delegate tasks to (useful for large investigations or specialized roles) ³. You connect **MCP servers** (e.g. Notion, database, Figma integrations) so Claude can interact with external services and data ⁴. You set up **Hooks** to run deterministic actions at certain events (for example, auto-run tests after each code edit) to enforce rules that the AI must not bypass ⁵ ⁶. Plugins are introduced as bundles of skills/hooks/subagents that can be installed for extra tools or shared functionality ⁷. By the end of L2, you’re wielding Claude Code’s full feature set: e.g. using a `/deploy` skill to deploy code, spinning up a “DesignReviewer” subagent to critique a Figma file, or having a PreToolUse hook block any attempt to edit sensitive files. You also learn to fine-tune **permission settings** – setting allowlists for trusted commands or using `claude /sandbox` for OS-level isolation ⁸ ⁹ – balancing speed with safety.

L3 – Automation and Scale: Level 3 is about scaling these capabilities for automation, CI/CD, and team workflows. Here you learn **headless mode** and continuous integration: running Claude Code in scripts and pipelines without an interactive chat ¹⁰. You integrate Claude with version control: e.g. GitHub Actions to have Claude automatically respond to PR comments or generate pull requests from issue descriptions ¹¹ ¹². You practice *fan-out patterns* – parallelizing Claude across multiple files or tasks for large-scale changes ¹³ ¹⁴. You create **plugin packs** or internal marketplaces to package your skills, hooks, and subagents for reuse across projects. The focus is on **CI-safe and enterprise-ready usage**: using least-privilege in automation (restrict Claude’s allowed tools in CI with flags like `--allowedTools` ¹⁴), storing API keys in secrets (never in repo) ¹⁵, and reviewing all AI-generated commits before merge. By the end of L3, you can deploy Claude-powered automations at scale – e.g. a commit hook that runs Claude to fix lint errors project-

wide, or a scheduled job that uses Claude to analyze weekly KPI data and post a summary (using a Slack MCP connector and a skill to format the report).

L4 – Extraordinary Workflows (Creative & Multi-Agent): The final level unlocks multi-agent orchestration and highly creative pipelines. You learn to coordinate multiple Claude instances and subagents working in tandem for complex projects. This might involve a “planner” agent that breaks a big task into parts and “executor” agents that tackle each part, or a writer vs. reviewer pair running simultaneously for higher-quality outputs ¹⁶. You'll design workflows that chain Claude with other AI or tools – for example, generating game narratives or dialog with one skill, then having another agent turn those into game code or assets automatically. You experiment with creative uses like using Claude Code to script life-automation routines (e.g. cross-referencing your budget spreadsheet and calendar to suggest cost-cutting, with one agent scraping data via MCP and another providing analysis). At L4, you also explore the boundaries: orchestrating Claude alongside other AI models or services, and implementing fail-safes for autonomous agents. **Multi-agent safety** is emphasized: you ensure each subagent has scoped permissions and context to prevent runaway behavior. Mastery at L4 means you can conceive a complex, multi-step workflow across domains (PM, game dev, personal finance) and implement it with Claude Code's framework – effectively using Claude as an *AI team of specialists* rather than a single assistant.

6-Week Mastery Syllabus and Learning Plan

Each week builds on the previous, with hands-on drills and both fast-track (solo) and enterprise-safe approaches. **Weekly format:** Goals → Drills (3-5) → Build artifacts → Success criteria. Enterprise-safe variants are noted for team settings (focus on least privilege, auditability, safe defaults).

Week 1: Setup, Fundamentals, and Everyday Usage

- **Goals:** Get Claude Code set up and practice core workflows for daily productivity. Learn to use the CLI and basic agent loop, create your first CLAUDE.md, and safely execute simple coding or automation tasks. Establish good habits with permissioning and context resets.
- **Hands-On Drills:**
- **Install & Update Claude Code:** Install the Claude Code CLI (or ensure you're on the latest version) ¹⁷. Log in and run `claude --version` to verify features are up-to-date. (*Enterprise-safe:* coordinate with IT if using an enterprise deployment or proxy).*
- **Hello World Project:** Initialize a new project (or a test repo) and run `claude` in it. Instruct Claude to “Create a hello-world script that prints a greeting” and step through the plan/execute cycle. Approve Claude's file write and run the code. This drill familiarizes you with the chat interface, Claude's planning, and permission prompts.
- **Create a CLAUDE.md:** In the project, create a `CLAUDE.md` with at least 3 guidelines: e.g. coding style or a “**NEVER do X**” rule about secrets. For example, “**NEVER** commit `.env` files or API keys” ¹⁸ ¹⁹. Restart Claude and confirm it loads your rules (Claude should mention following project guidelines).
- **Basic PM Workflow:** Prompt Claude with a product management task – e.g. “Draft a one-page PRD (Product Requirements Document) for a new login feature.” Let Claude generate the markdown or text, then refine it by asking for adjustments. Observe how Claude uses the context; practice using `/clear` between unrelated tasks to avoid context bleed ²⁰. Save the output to a file.
- **Personal Automation Script:** Ask Claude to automate a simple life task, like parsing a small CSV of expenses into a summary report. For instance: provide a small CSV of transactions, and prompt

“Write a Python script to read `transactions.csv` and total the expenses by category.” Approve file writes and run the script. Verify the output matches expectations. (*Enterprise-safe*: use only dummy data, and note that in a work setting you’d not expose real financial data to an AI without approval).*

- **Build Artifacts:** A functioning Claude Code installation; a basic `CLAUDE.md` in `~/ .claude/` (global) or project with your rules; example output files (the hello-world script, the generated PRD or spec in Markdown, the Python automation script). These files should be saved in your project repo (and `CLAUDE.md` checked into version control if team-shared).
- **Success Criteria:** By end of Week 1, you can comfortably start a Claude session, provide instructions, and get usable outputs. You’ve validated that Claude can create and modify files in your environment with your approval. You understand how to give clear prompts and when to reset context. *Test:* Describe a small task (e.g. “Add a comment to the script header”) and see if Claude follows it correctly. Verify your `CLAUDE.md` rule (e.g. the “NEVER commit .env” rule) by attempting to have Claude write a dummy secret – it should refuse or warn, confirming the rule is active.
- **Enterprise-safe Tips:** Always use real company projects in read-only mode at first. Verify Claude isn’t accessing confidential files unexpectedly (check the chat transcript for any file reads). Keep **permission mode** at default. If Claude attempts anything unexpected, use `/abort` or deny the action. Ensure your `CLAUDE.md` includes any mandatory org rules/policies (like coding standards or security mandates) provided by your org’s Claude admin.

Week 2: Advanced Everyday Workflows and Core Skills

- **Goals:** Deepen your everyday Claude usage with more advanced prompting and the introduction of **Skills** and context management. Learn to split larger tasks into steps (using Claude’s plans or your guidance), incorporate reference materials via skills, and streamline repetitive commands with improved permission settings. Build confidence in using Claude for both coding and non-coding PM tasks interchangeably.
- **Hands-On Drills:**
- **Skill Creation (Reference):** Create a simple reference **Skill** in `.claude/skills/`. For example, a skill named `team-styleguide` that contains your team’s writing or coding style rules. In `.claude/skills/team-styleguide/SKILL.md`, write a list of rules (e.g. “Product specs should start with an **Overview**, include **User Stories**, etc.”). Add a short frontmatter with a name and description. Use the skill by prompting `/team-styleguide` in Claude or letting Claude auto-load it when relevant ²¹ ²². Confirm that Claude pulls in the skill content when you ask a related question (e.g. “What are our style rules?”).
- **Skill Creation (Workflow):** Make an **action skill** to automate a routine. For example, a `budget-summary` skill that, when invoked as `/budget-summary`, will run a sequence to parse a finance CSV and output a summary. In `SKILL.md`, outline steps (you can include shell commands like “1. Read the file, 2. Calculate totals, 3. Generate Markdown report”). Include `disable-model-invocation: true` in the skill’s frontmatter if you want it to run only when explicitly called ²³. Test it by running `/budget-summary transactions.csv` within Claude. (*Solo-fast track*: you might let Claude run commands as part of the skill; *Enterprise-safe*: you might have the skill just output instructions for a human to run, if auto-executing is too risky).*
- **Interactive Planning:** Practice using Claude’s multi-turn abilities for a larger task. For instance, ask Claude to “Plan the steps to implement a simple text-based adventure game class.” Let it produce a plan. Then go step by step: have Claude execute step 1 (perhaps creating a file with class skeleton), approve it, then proceed to step 2, etc. This drill builds comfort with Claude’s internal planner and

using your judgment at each step (you can ask Claude for a plan with `/plan` or simply by prompting for an outline).

- **Permissions Tweaks:** Use the `/permissions` command or settings to reduce prompt fatigue. For example, allowlist a safe command: run `/permissions allow Bash("pytest")` to let Claude run tests without asking each time ²⁴. Or set up a project-specific `.claude/settings.json` with an allowlist for common tools (like your linter or `echo` commands). Test it by telling Claude to run that command and ensure it no longer asks for approval. (*Enterprise-safe:* coordinate with your team on an approved allowlist; never allow destructive commands. Consider enabling `/sandbox` mode to isolate Claude's file access to the project directory for safety ²⁵.)*
- **Context Management:** Deliberately push Claude's context limits to learn management. Feed a longer specification or multiple files, then ask questions to see how much Claude remembers. Practice using `/summarize` or instructing Claude to summarize parts of the context to stay within limits. If Claude's answers degrade after many turns, use `/clear` or spawn a new session. This drill teaches you to notice when Claude might be losing track, and reinforces the habit of modular, focused chats (one topic per session to avoid the "kitchen sink session" issue ²⁰).
- **Build Artifacts:** New skill files under `.claude/skills/` (at least one reference SKILL.md and one workflow SKILL.md). An updated `settings.json` in `.claude/` if you configured allowlists or other settings (e.g. `"permissions": { "Bash": ["pytest"] }` under allowlists). Any code or documents produced via the drills (e.g. the text-adventure game class file, the budget report output). These should be organized in your repo (you might create a `/skills` or `/experiments` folder to keep drill artifacts).
- **Success Criteria:** By end of Week 2, you are comfortable creating and using skills and managing Claude's context. **Test:** Invoke your reference skill by name in a fresh session and see if Claude follows those guidelines in its subsequent output (e.g. ask Claude to "Write a spec outline" and see if it uses your styleguide skill for format). Verify that your allowlist works by having Claude run a allowed command without prompting. You should notice improved efficiency (fewer "Allow?" prompts for routine tasks). Also, ensure you can recover from confusion: intentionally give Claude a slightly off-topic request mid-session and practice refocusing it or using `/clear`.
- **Enterprise-safe Tips:** In a team setting, store shared reference knowledge as skills in the repo so everyone's Claude has consistent info (Claude auto-loads skill descriptions at session start) ²⁶. Use managed policies if available (your org might enforce certain global CLAUDE.md content or block certain tools). Keep any allowlists scoped narrowly (specific commands only, not wildcards that could be abused). If you enabled sandbox, verify Claude truly can't write outside the project (e.g. ask it to read `/etc/passwd` to test the sandbox – it should be blocked ²⁷). Always review skill content for sensitive info (no secrets in prompt examples, etc.), since skills are part of the prompt when invoked.

Week 3: Hooks and Subagents for Enhanced Control

- **Goals:** Learn to use **Hooks** to enforce rules and automate actions, and **Subagents** to delegate tasks in isolated contexts. By the end of this week, you'll set up basic hooks (pre- and post- actions) and create custom subagent personas. This solidifies deterministic control (hooks run *every time* an event fires) and parallel thinking (subagents working without polluting main context).
- **Hands-On Drills:**
- **PreToolUse Hook – Safety Net:** Implement a simple PreToolUse hook to catch unsafe operations. For example, block editing of any `.env` file. In your `.claude/settings.json`, add a PreToolUse hook entry that matches the Write or Edit tool and calls a script (you'll write a short script to check the filename). *Hint:* The hook can read JSON input from Claude and exit with code 2 to block the

action ²⁸ ²⁹. For instance, if `tool_input.path` ends in `.env`, print a "BLOCKED" message and exit 2. Test by instructing Claude to open or edit an `.env` – it should be prevented by your hook, with your error message shown. (*Enterprise-safe*: This is a must – such a hook ensures even if Claude's policies fail, it **deterministically** stops secret leakage ³⁰ ³¹.)*

- **Post-Edit Hook – Automation:** Create a **PostToolUse** hook (or the `Stop` event hook) to run a quality check after Claude writes code. For example, after every edit operation or at conversation stop, run your linter and tests. In `settings.json`, configure a `Stop` hook that triggers on `"**"` (any stop) to execute a shell command like `npm run lint && npm test`. You can use a simple script or direct command in the hook config ³². Then have Claude make an intentional code change that breaks a lint rule, and observe that after the edit, the hook runs and outputs lint errors (which appear in Claude conversation). Claude should then attempt to fix them in response. This drill shows how hooks can create a feedback loop for Claude to self-correct (Claude will "see" the hook's stderr output as additional context).
- **Custom Subagent – Persona Creation:** Design a subagent persona file in `.claude/agents/`. For instance, create `design-assistant.md` with a persona that is a "Game Design Advisor" – it could have an emoji in the name (e.g. `# 🎮 Design Advisor` as title) and describe expertise ("Focus on creative storytelling and user experience"). Include a few expertise bullet points (like narrative design, level design feedback) ³³ ³⁴. Save it. Then in Claude, invoke this agent by name: e.g. prompt: `"🎮 Design Advisor, list three alternate story plots for the game level described in @design-doc.md."` Claude should switch to that subagent (likely indicated by colored name or a system message) and produce output in that persona. Confirm the subagent's style is distinct.
- **Subagent for Code Review:** Create another subagent, e.g. `security-reviewer.md`, with a persona focusing on code security analysis (expertise in spotting vulnerabilities). After defining it, use a main Claude session to delegate: "Use a subagent to review `auth.py` for security issues." Claude should spawn the `security-reviewer` subagent behind the scenes and return a summary of findings ³⁵ ³⁶. This drill teaches you the syntax to explicitly request subagent usage and shows how subagents read lots of content but return only salient info. (*If Claude doesn't auto-pick your custom subagent, you can explicitly refer by name as in the previous drill.*)
- **Hook & Subagent Combo (Optional):** For a challenge, combine hooks and subagents: e.g. a hook that on session start automatically runs a subagent to gather context. For example, when a session starts, trigger a subagent to read all recent commit messages and summarize them, feeding that summary to Claude at the outset (could be done by writing to a file that Claude reads or via the hook's stderr). Implement a `SessionStart` hook that calls a script to fetch recent git logs and prints a summary ³⁷ ³⁸. This ensures every new session begins with up-to-date context (useful for enterprise knowledge bases or daily stand-up notes).
- **Build Artifacts:** An updated `settings.json` file with at least one `PreToolUse` and one `Stop` (or `PostToolUse`) hook configured. Hook scripts (e.g. `block-secrets.py`, `end-of-turn.sh`) stored in a directory like `~/ .claude/hooks/` or your project (and referenced in settings). Two or more subagent Markdown files in `.claude/agents/` (e.g. `design-assistant.md`, `security-reviewer.md`), each defining the name, color, persona, and expertise sections. Logs or transcripts demonstrating the hooks firing and subagents responding (you might keep these as markdown notes).
- **Success Criteria:** By end of Week 3, you have enforced at least one critical rule with a hook and seen it work (e.g. Claude is **unable** to bypass the `.env` ban because your hook blocks it every time ³⁹ ⁴⁰). Your Claude environment now has "guardrails" that you designed. Also, you can create and invoke subagents at will. **Test:** If you run `/agents` in Claude, it should list your custom subagents ⁴¹. Try a realistic scenario: have your **Engineer** subagent (if you made one) review a piece of

technical spec text, and separately have your main Claude do it – compare the style of feedback. The Engineer subagent should give more technical depth if configured properly. Hooks test: intentionally attempt a risky operation (like writing a forbidden file or running `rm -rf` in a prompt) – confirm your PreToolUse hook catches or at least warns about it.

- **Enterprise-safe Tips:** Hooks are your best friend in enterprise settings – they provide **deterministic enforcement** ⁴² ⁴³. Work with security teams to develop hooks for company policies (e.g. blocking deletion of critical databases, or preventing any external network calls except through approved proxies). Store these hooks in the global config (`/etc/claude-code/CLAUDE.md` or a managed `settings.json`) for organization-wide protection ⁴⁴ ⁴⁵. For subagents in enterprise, limit their tool access: if the platform allows, configure each subagent's allowedTools (for example, your security-reviewer agent might only need Read access, not Write). Always review subagent outputs – they run in isolation, but if they summarize sensitive data, ensure they're not leaking it. Use subagents to compartmentalize sensitive tasks (e.g. one with access to financial database and another with access to code repo, so no single agent has all data). Keep persona files under version control so changes are auditable.

Week 4: Integrations, Plugins and MCP (Connecting to External Systems)

- **Goals:** Expand Claude Code beyond the local environment by connecting **MCP integrations** and exploring **Plugins**. Gain the ability to pull in data from or push actions to external services (APIs, databases, docs) and package your customizations for reuse. Emphasis this week is on **integration testing** – making sure Claude can safely interface with tools like Slack, Notion, or GitHub through its connectors – and understanding plugin architecture. Both individual productivity and team scenarios (enterprise) are considered in how you use or restrict these integrations.
- **Hands-On Drills:**
 - **Connect an MCP Server:** Use the CLI to connect an external tool. For example, if you have a Notion account, run `claude mcp add notion` (or another supported service like `claude mcp add jira` or a database) and follow prompts to authenticate ⁴⁶. Once connected, test a query: ask Claude via natural language to retrieve something, e.g. "Find the design spec in Notion titled 'Login Feature' and summarize it." Claude should use the Notion MCP to fetch content. Alternatively, connect a dummy database: if you have a local SQLite or a testing DB, use `claude mcp add database` and configure connection string. Then ask, "Query the database for the top 5 users by revenue." The key is seeing Claude use the MCP **tool invocation** (you'll notice Claude issuing a tool command like `Notion.query` or SQL query). (*Enterprise-safe:* use read-only credentials for any database; avoid connecting to production data. Ensure any integration you test is allowed by your company policy. For sensitive systems, skip this drill or use a safe sandbox copy.)*
 - **Use Community Plugin:** Install a plugin from the Claude community (or Anthropic marketplace). For instance, if you are coding in Python, you might install a **code navigation plugin** that improves Claude's knowledge of your code (e.g. a plugin that indexes symbols for jump-to-definition) ⁴⁷. Use `/plugin search` or a marketplace URL to find one (the **awesome-claude-code** list or official marketplace might have "code-intel" plugin). Install it with `/plugin install <name>`. Verify it's working by using its features (for example, after installing a review plugin, try `/review` to perform a code review ⁴⁸ ⁴⁹). Another example: use the "hookify" plugin to auto-generate hooks by describing them ⁵⁰. Practice uninstalling a plugin as well, to know how to remove unneeded extensions. (*Enterprise-safe:* Only install plugins from trusted sources. Prefer plugins your organization has vetted or that you've reviewed the code for. In enterprise settings, marketplaces can be restricted – you might instead use a company-internal plugin repository. Always inspect

plugin manifests and what components they add before installing, to avoid malicious hooks or commands ⁵¹ ⁵²).*

- **Build a Simple Plugin (Local):** Package one of your own skills/hooks into a plugin for reuse. For example, create a plugin called “BudgetBuddy” that bundles the `budget-summary` skill from Week 2 and maybe a subagent. Create a directory structure as per Claude’s plugin format (a `.claude-plugin` folder with a `plugin.json` manifest) ⁵³ . Write `plugin.json` with name, version, description, and list your skill under `skills/` . Optionally include a hook or agent in the plugin directory. Then test it locally by adding a local marketplace entry: run `/plugin marketplace add ./path-to-your-plugin-dir` and then `/plugin install budgetbuddy@local` (if using a local JSON). If done right, Claude will treat your packaged skill just like an installed plugin skill (namespaced if not a global plugin). This drill teaches you how to **reuse your automations across repos**. (If this is too involved, at least write the `plugin.json` and conceptualize the structure. In Week 5 we’ll formalize distribution.)
- **Enterprise Integration Scenario:** Simulate or plan an enterprise integration: for instance, design how Claude could update Jira tickets or post to Slack via MCP in a safe way. This can be a paper exercise or low-code: e.g. write a skill `notify-release` that uses a Slack MCP command to post a message when a release is cut. You might not actually send a Slack message if you can’t in your environment, but write out the steps/instructions in the skill (like “1. Use Slack MCP to post message to #releases channel”). Discuss or note how you would restrict this in enterprise (e.g. only allow posting to specific channels, require certain format). If possible, test with a personal Slack workspace to see Claude formulate a message via the MCP (ensuring you approve it). The goal is to think through **least privilege** – e.g. if connecting Claude to Slack, maybe use a bot token that only has access to one channel, etc., so even if prompt-injected it can’t abuse other channels.
- **Build Artifacts:** MCP connection configs (Claude typically stores these in a config file or keychain – ensure you document which services you connected and how). A list of installed plugins (maybe in a `plugins.md` or noted in your journal) including versions. Your custom plugin files: a plugin manifest (`plugin.json`) and any accompanying skill/hook files under that plugin’s structure. Any new skills created for MCP usage (like a skill documenting your DB schema or Slack message templates). If you wrote an integration skill or plan, include the `SKILL.md` with those instructions.
- **Success Criteria:** By end of Week 4, Claude Code is not limited to your local machine – it can pull in external info or trigger external actions under your guidance. **Test:** Ask a question that requires external data (e.g. “What’s the status of Design Task XYZ?” assuming that info is in a connected tool like Notion/Jira). Claude’s answer should demonstrate it used the integration (and the answer should be accurate to the external source). Another test: use an installed plugin command (like `/review` from a code review plugin) on a sample piece of code and see that it performs the intended enhanced behavior. For your custom plugin, verify that you can install it into a fresh project and the skill(s) appear (try `/budgetbuddy: budget-summary` if namespaced, for example).
- **Enterprise-safe Tips:** Treat MCP connections and plugins with extreme care. **Never hardcode credentials** – use the secure storage or secrets for API keys (Claude Code Actions documentation stresses using GitHub Secrets for keys ⁵⁴). Limit the scope of integrations: if Claude can query a database, consider making a read-only replica just for it, or limit queries to a certain schema. In enterprise deployments, you can use **managed marketplaces** – e.g. host your “Superuser Pack” plugin on an internal Git repo and have team members add that marketplace URL ⁵⁵ ⁵⁶ . This way, only vetted plugins are used. Also enable auditing: ensure Claude’s transcripts are logged or at least that actions taken via MCP are recorded (so if it posted to Slack or ran a DB query, there’s a trace). Finally, keep an eye on version compatibility: if a plugin uses features from a certain Claude version,

document that (e.g. “Requires Claude Code vX.Y for LSP server support”) so others can verify they meet prerequisites.

Week 5: Automation at Scale – CI, Headless Mode, and Pack Distribution

- **Goals:** This week focuses on scaling your Claude Code usage to many tasks and ensuring continuous integration setup. You'll integrate Claude into CI/CD pipelines (like GitHub Actions or GitLab CI) to automate code reviews, tests, or deployments. You will also finalize the **Superuser Pack** distribution: assembling your best skills, hooks, and configurations into shareable packs (for personal reuse across projects and for your team). The outcome is the ability to run Claude *unattended* on defined tasks and to have a portable toolkit of Claude customizations.
- **Hands-On Drills:**
 - **Claude Code in CI (GitHub Actions):** Set up Claude Code GitHub Action on a repository. Use either the quick installer (`/install-github-app` in Claude CLI) or manual steps ⁵⁷ ⁵⁸. Provide your Anthropic API key as a secret in the repo ⁵⁴. Create a workflow file (e.g. `.github/workflows/claude.yml`) that triggers on pull request comments. Use the official action (`anthropics/claude-code-action@v1`) with parameters to respond to `@claude` mentions ⁵⁹. For example, in a PR, comment “@claude run /review” and watch the Action have Claude analyze the PR changes and post a review comment. Experiment with another workflow: perhaps an issue labeled “automation” triggers Claude to attempt to implement that issue's description as a commit. *Verify* that Claude's changes appear on a new branch/PR and do not automatically merge (require human review) ⁶⁰ ⁶¹. This drill teaches how Claude can augment code reviews and development in a team setting. (*Enterprise-safe:* ensure the Claude GitHub App has minimal permissions – contents, issues, PRs as needed ¹², and absolutely do not give it access to secrets or environments. All AI suggestions should be treated as assistive, not final – require human approval in the workflow.)*
 - **Headless Batch Processing:** Use Claude's headless mode for a bulk task. For instance, if you have 50 markdown files of game dialogue and you want to find any that mention “dragon”, you could run Claude in a loop to analyze each file (though a simple `grep` might do, this is for practice). Write a small shell or Python script that does: `for each file in dialogs/, run: claude -p "Does this file mention 'dragon'? Respond with filename if yes." --output-format json`. Use `--allowedTools "Read"` to ensure Claude only reads files ¹³ ¹⁴. Run this script and confirm it prints results. Alternatively, use the fan-out example from docs: generate a list of tasks and have Claude attempt each in parallel processes. The key outcome is seeing Claude work without the interactive loop, producing output that your script can parse (e.g. JSON output). This could be applied to more complex tasks like migrating code patterns across dozens of files (Claude suggests a plan, then your script executes multiple Claude instances to apply changes concurrently).
 - **Enhance Pre-commit or Tests with Claude:** Pick a developer workflow pain point and automate it. For example, a pre-commit hook that runs `claude -p "Review my staged changes for TODO comments and remove them if present." --diff`. Simulate this by making a git pre-commit hook script that calls Claude (headless). Test committing code with a “TODO” comment and see if Claude catches and fixes it. Another idea: integrate Claude into your nightly build – e.g. after tests run, have Claude analyze the test failures and open a GitHub issue summarizing them (this could be done with a script calling Claude and then using GitHub CLI to create an issue, since Claude can output the summary). These automations solidify Claude as a CI tool.
 - **Finalize Superuser Pack (Starter/Power):** Gather all the artifacts you've created (skills, hooks, agents, settings). Organize them into a coherent folder structure (we'll specify in **Section C** below).

Make sure you separate what's generally useful vs. project-specific. For the **Starter Pack**, identify the minimal set of files that a new project or new user (you on a new machine) would need: probably a global `~/ .claude/CLAUDE.md` with your universal rules, a few universal skills (like your style guides or common tasks), and maybe global hooks for safety (the `.env` blocker can be global). For the **Power Pack**, include the more advanced/project-specific tools: specialized subagents, project-specific skills (like game-specific or finance-specific skills), and the fancier hooks (like test automation). If you haven't yet, try packaging these as an **internal plugin**: e.g. "Superuser Power Pack" plugin containing all your skills/hooks/agents. Document how to install it (maybe as simple as cloning a repo and adding with `/plugin marketplace add`). This drill readies your kit for distribution.

- **Enterprise-safe Pack Adjustments:** Now consider what the **Enterprise Pack** variant would look like: likely remove or modify any risky defaults. For example, if in your Power Pack you had a hook that automatically approves certain actions, the enterprise version should *not* auto-approve anything except perhaps trivial ones, and instead favor sandboxing. Adjust permission config to be more restrictive (maybe a denylist of dangerous commands in `settings.json` on top of allowlists ⁶²). Ensure auditability: e.g. a hook that logs every Claude action to a file or monitoring system. If possible, implement a simple logging hook (maybe a `PostToolUse` hook that appends `tool_name` and `tool_input` to a `actions.log` file for each tool usage). While you might not deploy this fully now, outline the changes (you can create a separate `enterprise-settings.json` file with these stricter rules).
- **Build Artifacts:** CI workflow files (e.g. `.github/workflows/claude.yml`) in a test repo. Scripts for headless usage (shell or Python scripts you wrote for batch processing or pre-commit hooks). The organized **Superuser Pack** directories:
 - A **Starter Pack** folder (or documentation) containing baseline files like `CLAUDE.md` template and must-have hooks/skills.
 - A **Power Pack** (your full collection of advanced `.claude` configs, possibly as a plugin or a zip of the `.claude` directory).
 - An **Enterprise-safe Pack** documentation or config overrides (like a hardened `settings.json`, notes on what not to include, how to deploy the pack internally). Additionally, update your README or documentation to explain how to use each pack. This might include a "How to install" section for your plugin or instructions to copy files into one's `~/ .claude/`.
- **Success Criteria:** By end of Week 5, you've run Claude outside the comfort of the chat UI – in automated jobs and scripts. **Test:** Trigger your GitHub Action by commenting on a PR and confirm Claude's response comment is posted (and is accurate). Check that your pre-commit hook or script successfully invoked Claude (e.g. the commit goes through with Claude's modifications, or the script's output is correct). For the packs, simulate a fresh start: on a new project, apply your Starter Pack (copy in the `CLAUDE.md` and hooks) and ensure Claude works with them; then apply Power Pack extras if needed. In an imaginary scenario, hand off the Enterprise Pack to a colleague – would they understand how to install it and would it default to safe behavior? If yes, you've documented it well.
- **Enterprise-safe Tips:** When integrating into CI, **set strict timeouts and resource limits** – e.g. `--max-turns` or job timeouts so that if Claude gets stuck, it doesn't run forever ⁶³ ⁶⁴. Only allow Claude to commit to a separate branch, never directly to `main`. Use protected branches so AI changes require review. In your enterprise pack, consider including a **policy file** if Anthropic provides one for team-wide rules (the global `CLAUDE.md` at org level) ⁴⁴. Also ensure that the pack is easy to update – for example, host it in a repo so updates can be pulled, and perhaps use version numbers. If possible, perform a **dry run** of the pack in a staging environment: have a colleague use your

Power Pack on a dummy project and gather feedback, especially on security concerns. This will validate its enterprise readiness.

Week 6: Creative Multi-Agent Orchestration and Final Project

- **Goals:** In the final week, put everything together in an extraordinary project that showcases creative and complex use of Claude Code. Emulate a real-world scenario in one (or more) of your domains (product management, game development, life automation) that leverages multi-agent orchestration, parallelism, and the full range of Claude's capabilities. The goal is to push Claude Code to its limits while maintaining control. By the end, you will have a "capstone" workflow and a refined Superuser Pack (v1) ready for use.
- **Hands-On Drills:**
- **Capstone Project Planning:** Choose a substantial project that excites you and spans your interest areas. For example: *"AI-augmented Game Level Design"* – where Claude helps generate a game level storyline, writes some code, and produces test cases; or *"Automated Product Requirements to Prototype"* – where Claude takes a high-level product spec and generates a prototype code + summary slides; or *"Personal Finance Planner"* – where Claude aggregates data and produces a financial plan. Write down the project goals and break it into stages. **Assign roles** to agents: e.g. a "Planner" agent to outline tasks, a "Builder" agent to code, a "Tester" agent to validate, a "Presenter" agent to compile results. This plan itself is an artifact (consider storing it in a markdown file or as a skill).
- **Multi-Agent Execution:** Implement your plan using multiple Claude sessions or subagents. For instance, start with your **Planner** (which could just be main Claude in a fresh context or a dedicated subagent) to produce a task breakdown. Then, in parallel (if possible) or sequentially, run the **Builder** steps: you might open two terminals, one with Claude focusing on coding, another with Claude (perhaps as a different subagent) focusing on creative content. A practical way: have Claude use the `Task` tool to spawn subagents for certain tasks ³⁵ – e.g. "Claude, use subagents to research these 3 topics and report back." If subagents aren't fully autonomous enough, simulate by manually running separate Claude instances. The key is orchestrating: ensure each agent knows its role (via your instructions or using the persona files). Use hooks or scripts to coordinate (maybe a simple script that triggers the next agent once one's output is ready). This drill might be iterative and require troubleshooting to get all pieces to cooperate.
- **Creative Pipeline Drill:** If your project is creative (like narrative or design), leverage Claude's strength in natural language. For example, for a game storyline, have Claude generate several characters and dialogues (creative writing skill), then have another agent turn those into structured game data (coding skill). Use file passing: one agent writes to a file (like `story.txt`), then another agent reads that file and does something (like convert story to JSON format for game ingestion). Practice using Claude's ability to chain through the filesystem or via explicit commands. This mimics how multiple specialized AIs might work together on a pipeline.
- **Final Hooks/Automation:** Augment your final project with any remaining hooks or automation to fully close the loop. For example, if the final step of your project is presenting results, you could have a hook that automatically opens the generated report (perhaps using an MCP to open a browser or send an email). Ensure these automations are safe: e.g. for a demo, you might enable a hook to use `xdg-open` (on Linux) or `open` (on Mac) to show a file, but in enterprise, you'd replace that with a manual review step. This is an exercise in knowing when to automate and when not to.
- **Review and Retrospective:** Finally, perform a thorough review of your 6-week journey. Use Claude itself to help: ask it to analyze your `~/ .claude/` folder and list all the customizations you've made (skills, agents, hooks). Verify each is documented and necessary – remove or refine anything

redundant (e.g. maybe you have overlapping skills that could be merged). Check for **version-specific notes**: e.g. if you used a feature that's experimental or changed recently, note to verify it when versions update (Claude Code may update frequently; for instance, ensure your hooks still work if the schema updates, by reading the changelog). This retrospective drill ensures your knowledge solidified and your environment is clean going forward.

- **Build Artifacts**: The completed **capstone project** artifact – this will vary but could be a set of files (code, documents, outputs) produced by Claude under your orchestration. For example, a folder with game storyline drafts, code files, test results, and a summary README that was partly written by Claude. Any orchestration scripts or notes you used (if you wrote a shell script to coordinate agents, include that). **Final Superuser Pack v1** assembled: this might now include refinements you added in week 5 and any new creative skills or agent personas from the capstone. Version this pack (e.g. tag it v1.0) – this could simply be a dated folder or a git tag in your repository of Claude config. A short **Changelog** or README for your pack is a good idea, summarizing what each component does (for yourself and others).
- **Success Criteria**: By end of Week 6, you have demonstrated mastery by successfully executing a complex, multi-faceted workflow with Claude Code. **Evaluation**: Did your capstone project achieve the desired outcome? For example, if it was “PRD to Prototype,” do you have a working prototype and a document of requirements that align? If it was creative, did the output surprise and satisfy you (an indicator that you pushed Claude to be creative but contained)? You should be able to answer: how would you deploy such a workflow reliably if you had to repeat it – that means you’ve documented the steps and perhaps automated parts of it. Ideally, you have something you could present (to your team or in a blog) about what Claude Code enabled you to do.
- **Enterprise-safe Tips**: For a production-like multi-agent orchestration, consider using **checkpoints** or manual approvals between stages. In a company, you might never let an AI chain complete from planning to coding to deployment with no human in loop at some stage. Decide where those checkpoints should be (maybe after planning and after coding). Implement “gates” – e.g. your hook from week 5 that logs actions could also require a human to set a flag in order for the pipeline to continue. In terms of audit, make sure your final workflow outputs logs for each agent's activity (which subagent read which files, what decisions were made). With your pack v1 ready, also plan how to maintain it: in an enterprise, you might propose an **internal wiki or repo** for “Claude Code Best Practices” where your syllabus learnings and pack files reside for others to learn. Consider this the beginning of continuous improvement – your pack and skills should be revisited as Claude Code evolves (stay tuned to Anthropic’s changelog and community forums for any breaking changes or new features that could enhance or affect your setup).

Superuser Pack v1 Specification

Overview: The **Superuser Pack** is a curated collection of Claude Code configurations, templates, and tools that you’ve developed to boost productivity. It is organized into three tiers for different needs: a Starter Pack (essentials), a Power Pack (full feature set for solo use), and an Enterprise-Safe Pack (hardened for team use). Below is the folder structure, contents of each file, and notes on reusability versus project-specific elements.

```
superuser-pack/  
├── starter-pack/  
│   └── CLAUDE.md  
# Global/project-neutral conventions (no
```

```

sensitive info)
|   └─ settings.json           # Basic settings (safe permissions, no
dangerous skip)
|   └─ skills/                # Essential skills
|       └─ team-styleguide/    # e.g. writing or coding style guide skill
|           └─ SKILL.md
|       └─ commit-checklist/   # e.g. commit message checklist skill
|           └─ SKILL.md
└─ power-pack/
    └─ CLAUDE.md               # Extended CLAUDE.md (may include personal
workflow rules)
|   └─ settings.json           # Advanced settings (allowlists, maybe skip
in sandbox)
|   └─ skills/                # All skills from starter plus advanced ones
|       └─ ...
|   └─ agents/                # Custom subagents definitions
|       └─ design-assistant.md
|       └─ security-reviewer.md
|       └─ ...
|   └─ hooks/                 # Hook scripts for automation and guardrails
|       └─ block-secrets.py    # Example: blocks secret file access 28
|       └─ post-edit-tests.sh  # Example: runs tests on Stop event
|       └─ ...
|   └─ plugin/manifest/       # (Optional) plugin packaging for this pack
|       └─ marketplace.json    # If distributing via marketplace
|       └─ plugin.json         # Defines the Superuser plugin (bundling
skills, etc.)
└─ enterprise-pack/
    └─ CLAUDE.md               # Organization-specific rules (policies,
NEVERs)
    └─ settings.json           # Locked-down settings (deny lists, strict
prompts)
    └─ skills/                # Sanitized skills (no proprietary info
unless needed)
    └─ agents/                # Maybe fewer agents or ones with limited
scope
    └─ hooks/                 # Hooks focusing on security (e.g. logging,
approvals)

```

Starter Pack (Tier 1) – Essentials: The starter-pack is meant for a new Claude Code user or new project to get off on the right foot. It includes a template **CLAUDE.md** with general best practices and “always/never” rules (for example, a condensed version of your global rules like never commit secrets, always write tests, branch naming conventions, etc.). It’s kept broad and non-project-specific so it can apply everywhere. The **settings.json** here is minimal – it might explicitly turn on safe defaults (like requiring permission for all tools, no auto-skips) and possibly set a default sandbox policy if desired. The **skills/** in starter are limited to universal knowledge: e.g. a style guide, a common checklist (like commit message standards or code review

checklist). These skills are reusable across any repo (they don't reference specific project details). There may also be a generic "daily summary" skill or "standup report" skill for PMs that can be adapted anywhere. The Starter Pack does *not* include any custom agents or heavy automation – it's focused on context and guidance. *Reusable*: Everything in starter-pack is meant to be reused across repos. In fact, you might place these files in your home `~/ .claude/` so they load for all projects ⁶⁵. The starter pack forms the baseline that both power and enterprise packs build on.

Power Pack (Tier 2) – Full features for personal use: The power-pack contains the complete set of your Claude Code enhancements developed in Weeks 1–5. It builds on starter-pack by adding all the **agents** (subagents) you created, the advanced **skills** (including project-specific ones like game design templates or budget analysis workflows), and all the **hooks** for automation. The `CLAUDE.md` here might be an expanded version that includes personal workflow preferences (e.g. "Always prefer functional components over classes in React" if that's your style, or "Use my GPT prompt formatting style for creative writing" – anything that's your personal touch). **settings.json** in power-pack is tuned for speed and convenience: for example, it might whitelist a bunch of safe commands (tests, lint, git operations) so you're not prompted every time, and if you're comfortable, it might enable `--dangerously-skip-permissions` *but only in a sandbox* as a quick toggle for certain tasks (though use caution with this) ⁶⁶. The power pack's content is more project-specific: some skills might only apply to your game dev projects or to your finance scripts. You should note which skills are generic vs. which assume a certain project structure. The **agents** defined (like `design-assistant.md`, `security-reviewer.md`, etc.) are semi-generic – they represent roles that could be useful in multiple projects (e.g. any project can use a security reviewer). The **hooks** here automate your frequent tasks: e.g. a hook to run tests on file changes, a hook to format code, a PreToolUse hook to enforce policies. These might be tuned to your personal workflow (for instance, if you always use `pytest`, the hook is set up for that). *Reusability*: Parts of the power pack are widely reusable (especially if packaged as a plugin), but some may need tweaking per project. For example, a `deploy` skill might contain steps specific to one application's deploy process – that would need editing before using on a different app. When sharing or moving to a new project, you can include all power-pack files and then selectively disable or adjust ones that don't apply.

Enterprise-Safe Pack (Tier 3) – Team-safe automation: The enterprise-safe pack is a variant of your power pack adjusted for multi-user, regulated environments. Its emphasis is on **least privilege** and **auditability**. The `CLAUDE.md` here likely includes company-wide rules or compliance-related instructions (for example, rules about handling PII, code quality standards that everyone must follow). It could also remove any personal informal instructions and instead include formal policy (possibly provided by your org's engineering leadership). The **settings.json** is heavily restrictive: it could include a **denylist** of commands that are too dangerous (e.g. `docker push` might be denied unless specifically allowed, or `rm -rf` completely blocked) ⁶². It might enforce always sandboxed mode by default for all users. Your hooks focusing on safety (like the secret-blocker, logging hooks, etc.) are definitely included. Hooks that were convenience (like auto-fixing lint) might be toned down to only *suggest* fixes (to avoid Claude making unseen changes). The **skills** included in enterprise pack should be those beneficial to all (or most) team members – e.g. a skill encoding the team's Definition of Done or a skill for the company's API guidelines. Any skill that encodes proprietary info should be carefully reviewed (maybe the enterprise pack will actually strip out certain things if sharing outside a specific project – you might maintain separate packs per department to avoid cross-project data mixing). **Subagents** in enterprise pack might be limited or standardized personas (like the Engineer, Executive, QA personas from the PM guide) that management has approved for broad use. Possibly their allowed tools are configured at a policy level (ensuring, say, an "Executive" subagent cannot write code, only read and summarize). *Reusability*: The enterprise pack is

meant to be shared across many repositories in the company. It might be distributed via a **private plugin marketplace** ⁶⁷ ⁶⁸ for ease of installation – for example, the IT department can host a `marketplace.json` listing the enterprise-approved plugins (which could just be the enterprise-pack itself packaged as a plugin). Team members would then do `/plugin marketplace add <Corp URL>` and `/plugin install enterprise-pack@1.0`. Alternatively, the files could be placed by a bootstrap script on each repo (though that's less convenient to update). The key is that the enterprise pack has clear separation between what's global (applies to all projects) and what can be extended in specific projects. You may document which parts of it can be overridden. For instance, a project might have its own `CLAUDE.md` in addition to the global – the combination yields the final instructions (Claude merges them, with specific > general precedence ⁶⁹).

Templates and Content of Files: In all packs, each file follows the Claude Code specification: - **CLAUDE.md:** Markdown with project or global instructions. Your provided templates likely include sections for coding guidelines, workflow rules, and absolute “NEVER do X” rules. For example, the enterprise CLAUDE.md might have: “**NEVER expose customer data in output. If code or conversation contains <SECRET>, redact it.**” (with explanation that Claude should follow these rules). - **SKILL.md files:** Each skill has a YAML frontmatter with `name` and `description`. The content is either a reference (guidelines, documentation) or an action workflow (ordered steps). You have templates for both types. E.g. *Reference skill template*: a title, then bullet points or sections of reference info (like Part 7 of the guide suggests, treat it like an onboarding guide ⁷⁰ ⁷¹). *Workflow skill template*: a list of steps possibly with placeholders (you used this in budget-summary or commit-checklist). These are written in plain language for Claude to follow when invoked. - **Agent files:** As created in week 3, these are Markdown with specific structure (Name, Color, Persona, Expertise). Your pack might include some base templates for common roles (Engineer, Designer, etc. with emoji) to encourage consistent usage across team. Each agent file can be copied or modified by others easily since they're just text personas. - **settings.json:** This JSON holds configuration. Your templates may include commented sections or examples for common configurations: e.g. how to set up PreToolUse hooks, what an allowlist entry looks like, etc. For instance, a snippet in your enterprise settings might look like:

```
"permissions": {
  "allow": ["Bash(git diff)", "Bash(pytest *)"],
  "deny": ["Bash(rm *)", "Bash(curl http*)"]
},
"hooks": { ... }
```

with appropriate hooks definitions. We saw an example of hooks configuration in JSON where hooks are listed by event with matchers ⁷² ³². - **plugin manifest:** If packaging, `plugin.json` defines *name*, *version*, *description* and implicitly includes the skills/hooks/agents in that directory. (Your pack likely is one plugin with multiple components – as shown in the plugin example, you just need to bundle them and list in marketplace file for distribution ⁷³.)

Reusable vs Project-Specific: A guiding principle in your pack structure is to keep truly reusable stuff at the top level, and project-specific customizations in sub-folders or separate files. For example, your **Starter Pack CLAUDE.md** could be used in any project (it has no project names or proprietary details), whereas a skill like `api-conventions` might be specific to backend projects. You might mark such skills with comments or naming (e.g. prefix with project name if needed). Encourage usage of project-level CLAUDE.md and skills for anything that doesn't make sense globally. In pack distribution, you might provide

template files for those, e.g. a `PROJECT_CLAUDE.md.template` that teams can fill in their specifics, on top of the global pack.

Versioning and Maintenance: Superuser Pack v1 is your initial release. In your spec, note any version-sensitive aspects: e.g. "Tested on Claude Code vX.Y.Z – hooks configuration may change in future versions, check official docs ⁷⁴ if a hook does not behave as expected." If Anthropic releases updates (say Claude Code v3 with new hook events or changed context limits), you should plan to update the pack accordingly. Perhaps maintain a `CHANGELOG.md` in your pack to track changes over time. This ensures that as you or others use it across repos, they can update safely.

Risks and Mitigations

Using Claude Code's advanced features comes with important risks. Below are key risk areas and how to mitigate them, especially in an enterprise context:

- **Permission & Privilege Risks:** By default Claude Code asks permission for file writes, shell commands, and external tool calls – this is a crucial safety net. Bypassing it (with allowlists or `--dangerously-skip-permissions`) can speed you up but introduces risk of the AI executing harmful actions. **Mitigations:** Always apply *least privilege*. Use **allowlists** to permit only specific safe commands (and ideally with wildcards constraints, e.g. allowing `npm test` but not arbitrary `npm` commands). For any destructive operation (deploys, DB migrations), require explicit permission each time or even a human double-check. If you do use `--dangerously-skip-permissions`, do so only in a controlled environment like a disposable container or sandbox VM ⁷⁵. Never run skip-permissions on your actual production system. In enterprise settings, prefer **sandboxing** Claude at the OS level (Claude's `/sandbox` feature) which restricts file system and network access so even if Claude tries something sneaky, it's confined ⁹ ⁷⁶. Additionally, maintain a **denylist** for particularly sensitive commands (e.g. never allow `rm -rf /` or networking commands to unknown hosts) in the global settings, so even if an engineer whitelists broadly, the denylist overrides (enterprise admins can enforce this in managed config).
- **Prompt Injection & Tool Injection:** Claude, like any LLM, can be susceptible to malicious instructions hidden in content (prompt injection). An attacker could embed a command in a file or ticket that causes Claude to execute unwanted tools or reveal info. We also saw that if an unvetted plugin is installed, it could include a hook to auto-approve dangerous commands ⁵² ⁷⁷. **Mitigations:** Treat external inputs with zero trust. Use **PreToolUse hooks** as a final gate – for example, a hook that intercepts any Bash command and vets it against rules or even prompts the user again if it looks suspicious (this is effectively a custom confirmation layer). Keep Claude's **context narrow** – avoid letting it read large swathes of untrusted data in one go; if it must, use subagents to sandbox that reading and then have them summarize (reducing the chance that a hidden instruction makes it back to the main agent). Only install **trusted plugins/marketplaces** – e.g. those from official sources or your internal marketplace. Review plugin manifests and code: since plugins can contain hooks, subagents, etc., inspect them for any `--dangerously-skip-permissions` usage or odd hooks. An enterprise mitigation is to run Claude in a mode where it ignores new plugins unless signed by the company (if such a feature exists or via policy). Monitor network commands: if Claude has the ability to use `curl` or similar, ensure outputs are monitored

(to catch if it's exfiltrating data) – or simply block networking commands entirely except to whitelisted domains.

- **Hook Hazards (Infinite Loops, Unintended Consequences):** Hooks are code you write, so bugs there can cause issues like loops (e.g. a hook triggers, modifies something that triggers the hook again). A poorly thought-out hook might also block Claude in ways that are hard to debug (imagine a hook that always exits with code 2 due to a bug – Claude would be unable to execute certain tools at all). **Mitigations:** Test hooks thoroughly in isolation. Use conditions in your hooks – e.g. if you have a Stop event hook that runs tests, ensure it doesn't run if tests are already running or ensure it exits quickly to not hog the cycle. Claude provides hook exit codes to handle different outcomes (0 = proceed, 1 = error but continue, 2 = block) ²⁹ – use them correctly. For example, if your hook script itself crashes, have it fail open (exit 0) rather than accidentally blocking everything. Put safeguards to prevent loops: if you have a hook that commits changes, be careful, as that commit could trigger another Claude action – perhaps disable certain hooks during automated runs (you could set an env variable when running a script via hook to tell other hooks to skip). Documentation and small logical steps help – keep each hook focused (one for blocking secrets, another for running tests, rather than one hook doing too many things). In enterprise, consider **managed hooks**: an admin-curated set of hooks that developers can opt into but not modify, to ensure reliability. And log your hooks' actions – e.g. have hooks print to a log file, so if Claude is acting weird, you can check if a hook misfired.
- **MCP and External Access Risks:** When Claude can access external systems (databases, third-party APIs, Slack, etc.), it inherits those systems' risks. For example, if connected to a database with write access, a bad prompt could wipe a table. If connected to Slack, Claude could post confidential code to a public channel by mistake. **Mitigations: Credential management** is paramount – use read-only or minimal scopes for API keys you provide to Claude (e.g. if Claude needs to query a DB, give it a user account that can only SELECT, not DROP or DELETE). Keep a tight leash on what Claude is allowed to do via MCP: many MCP connectors might have their own config (for instance, if you use a HTTP request MCP, restrict allowed domains). Some teams even build a proxy for AI access – for example, instead of giving direct DB access, have an intermediate service that only exposes certain safe queries. Monitor usage: enable logging on those external services to track what queries or actions Claude took. For highly sensitive actions (like spending money via an API, or sending emails), require a human confirmation outside of Claude (e.g. have Claude draft the email but a person sends it). Always review **the first few times** Claude uses a new integration to ensure it's doing what you expect. On authentication: do not put API keys in CLAUDE.md or prompts (Claude might accidentally expose them); store them in Claude's secure storage or environment where it can use them without "seeing" them. If using Claude Code in a platform like AWS/GCP (via Bedrock/Vertex), ensure IAM roles are scoped to least privilege as well ⁷⁸ ⁷⁹ (so Claude's Actions in CI can only access specific resources).
- **CI/CD and GitHub Action Safety:** Running Claude in CI means it could potentially alter code or configuration automatically. Risks include it making an incorrect change that breaks the build or, worse, introduces a security flaw, and that change being automatically deployed if not caught. Also, if not isolated, CI runs could expose sensitive environment variables to Claude. **Mitigations:** Use **separate Claude API keys for CI** with limited scope if possible (Anthropic keys might not have scopes, but treat that key as highly privileged and don't reuse it elsewhere). In your GH Actions, **never pass secrets directly to Claude** – e.g. don't prompt Claude with secret values. Keep Claude's operation environment minimal (the GH Action by default keeps code on GitHub's runner, which is

good ⁸⁰ – ensure no secret files are in the repo that Claude can read). Set timeouts and max-turns for CI Claude runs to avoid runaway costs or infinite loops ⁸¹ ⁶⁴. For example, if using `anthropics/claude-code-action`, configure `claude_args: "--max-turns 5"` or a sensible limit so it doesn't loop forever on a hard problem. Use concurrency limits – e.g. don't run 10 Claude jobs in parallel unless you intend to (can burn through tokens and also possibly conflict). Critically, **require human review** for code changes: we recommend the Action opens a PR with changes rather than pushing directly to main, and do not have that PR auto-merge. If Claude is set to respond to comments, maybe avoid letting it respond to its own PR without a person initiating. Also, follow GitHub's principle of least privilege for the Claude app: it only needs Contents (code) write to create commits, and maybe Issues/PRs read/write ¹² – it should not need admin or repo settings access. You can even limit which branches it can write to. Regularly rotate the API key and remove Claude from the repo if not needed to ensure nothing lingering. Lastly, treat Claude's suggestions in CI as you would an unknown contributor's PR – do code review on them. Many of these safeguards are implied, but it's good to explicitly institute them.

- **Version-Specific Features and Verification:** Claude Code is evolving; features like Hooks, certain MCP integrations, or multi-agent enhancements might be in beta or change syntax over time. There's a risk your skills or config break after an update. **Mitigations:** Keep an eye on the official **Changelog** ⁸² ⁸³ and community forums. Before you upgrade Claude Code (especially in enterprise where you might batch upgrade for many users), test your critical workflows on the new version (perhaps in a staging environment or with a canary user). For example, if a new version changes how skill descriptions load, verify your skills still trigger as expected. Use feature flags or version checks in your config where possible: e.g. if a hook event name changed, you might temporarily register both old and new event names until all are on the latest version. When writing skills or prompts, avoid relying on undocumented behaviors that could change – stick to documented APIs (for instance, use the official hook events and not assume any internal variable beyond what docs specify). In your Superuser Pack docs, list the Claude Code version it's compatible with (e.g. "Tested on Claude Code 2.0 (Opus 4.5 engine) as of 2025-11"); update this as you test new versions. If you encounter a bug that you suspect is version-related, use the `/version` command or check `claude --version` to confirm environment, then reach out to support or check the changelog for any breaking changes. This due diligence will save headaches when, say, Claude Code 3.0 comes out with a different approach to plugins or context window sizes.
- **Data Privacy and Compliance:** Particularly for enterprise, any AI usage must consider privacy. Claude reading source code or user data might be sensitive. Anthropic promises certain privacy and compliance features ⁸⁴, but you should still enforce your own. **Mitigations:** Anonymize data before feeding to Claude when possible (e.g. replace real user names/emails with placeholders in prompts). Use Claude's tools rather than raw data: for example, instead of showing Claude an entire `.env` with secrets, use a hook to have a sanitized version or just have Claude ask for what it specifically needs. If needed, opt for on-prem or Claude on your cloud (Bedrock/Vertex) to keep data within your environment ⁸⁵ ⁸⁶. Make sure all colleagues using Claude Code are aware of what they can or cannot ask it to do (maybe your pack's `CLAUDE.md` includes a reminder not to reveal customer data). Log all Claude interactions if required by compliance, and inform users (via a policy banner or so) that those logs exist. Basically, treat the AI like a junior engineer: it needs access to do its job, but you wouldn't give a junior unrestricted prod DB access or let them publish things without review – the same caution applies.

Next Actions This Week

To put this learning plan into immediate practice, here's a checklist for **Next Actions This Week** (Week 1 getting started):

- 1. Install Claude Code & Verify Access:** If you haven't already, install the Claude Code CLI using the official script or package (e.g. Homebrew on Mac) ¹⁷. Launch `claude` in a sample directory and log in to ensure you can start a session. Update to the latest version (`claude update` or equivalent) so all features are available.
- 2. Set Up a Practice Repo:** Create a new project or choose a low-stakes repository (e.g. a personal project) to be your Claude Code playground. Initialize a git repo and add a basic file to work with. This will be used for trying out the drills (so you don't risk real work during learning).
- 3. Create Your Global CLAUDE.md:** Draft a `~/.claude/CLAUDE.md` with at least 2-3 key rules or notes you want Claude to always remember. For example: "Always write clear commit messages" and "Never disclose API keys." Save it and restart Claude to confirm it loads. This lays the foundation for consistent behavior ⁸⁷.
- 4. Complete Week 1 Drills:** Go through the Week 1 exercises above in order. For each drill, take brief notes on any issues or questions that arise. Especially note how permission prompts work and how to approve/deny confidently. By end of the week, ensure you have the Hello World script, a sample PRD generated, and the budget script tested. These will reinforce core usage.
- 5. Plan Weeks 2-6 (Schedule & Resources):** Carve out time on your calendar for the subsequent weeks. For each upcoming week, list any setup needed (e.g. for Week 3, you might need ideas for subagent personas; for Week 4, have API keys ready for integrations). Also bookmark the Claude Code documentation pages (Skills, Hooks, etc.) for quick reference. If you have teammates, consider a short "demo session" at end of each week to share progress – teaching others will reinforce your mastery.
- 6. Enterprise Alignment:** If you aim to apply this at work, initiate a conversation with your manager or IT about using Claude Code. Check if there's an existing policy or if you need to request permissions (for example, installing the Claude GitHub app for your team). Ensuring your company is on board early will make the enterprise-safe track smoother. Share a brief of this roadmap with relevant stakeholders so they know you're building with security in mind (they'll appreciate the proactive stance on hooks and permissions).

By following the above steps, you'll be well on your way to Claude Code mastery. Good luck, and enjoy the journey of augmenting your development and PM workflows with AI assistance!

¹ ²⁶ ⁶⁹ Extend Claude Code - Claude Code Docs

<https://code.claude.com/docs/en/features-overview>

² ¹⁷ ⁸⁴ Claude Code overview - Claude Code Docs

<https://code.claude.com/docs>

3 4 7 8 9 10 13 14 16 20 21 22 23 24 25 27 35 36 46 47 66 75 76 **Best Practices for Claude Code - Claude Code Docs**

<https://code.claude.com/docs/en/best-practices>

5 6 18 19 28 29 30 31 32 37 38 39 40 42 43 44 45 62 65 70 71 72 87 **The Complete Guide to Claude Code V2: CLAUDE.md, MCP, Commands, Skills & Hooks — Updated Based on Your Feedback : r/ClaudeAI**

https://www.reddit.com/r/ClaudeAI/comments/1qcwckg/the_complete_guide_to_claude_code_v2_claudemd_mcp/

11 12 15 54 57 58 59 60 61 63 64 78 79 80 81 82 83 85 86 **Claude Code GitHub Actions - Claude Code Docs**

<https://code.claude.com/docs/en/github-actions>

33 34 41 **1.5: Custom Sub-Agents – Claude Code for Product Managers**

<https://ccforpms.com/fundamentals/custom-subagents>

48 49 53 55 56 67 68 73 **Create and distribute a plugin marketplace - Claude Code Docs**

<https://code.claude.com/docs/en/plugin-marketplaces>

50 **The Shorthand Guide to Everything Claude Code**

https://x.com/giyu_codes/status/2012420750855012428

51 52 74 77 **Hijacking Claude Code via Injected Marketplace Plugins**

<https://www.promptarmor.com/resources/hijacking-claude-code-via-injected-marketplace-plugins>